

DETECTION ENGINEERING AND ALERT TRIAGE

REPORT

BY

UTIBE ANTHONY

Executive Summary:

This project focused on validating, improving, and demonstrating the effectiveness of a previously developed Security Operations Center (SOC) lab environment. The work was conducted across multiple phases, beginning with the validation of existing log sources and progressing through detection engineering, alert validation, and simulated attack testing.

During Phase 1, the existing SOC lab environment was reviewed to determine whether the major log sources used in the previous SOC/SIEM lab were still active and providing usable telemetry. The validation covered Sysmon logs, Windows Security logs, Linux logs, and firewall logs. Existing gaps identified from the previous project were also reviewed, with attention given to whether the required telemetry was reaching the SIEM and whether the environment was still suitable for security monitoring and detection activities. Where gaps were identified, configuration adjustments were made to restore or improve telemetry collection.

Phase 2 focused on detection development. Five detection rules were developed and mapped to relevant MITRE ATT&CK techniques, covering activities such as PowerShell execution, Scheduled Task/Job activity, SSH remote services, Local Account Discovery, and Process Discovery. The rules were implemented in Wazuh and designed to generate alerts when corresponding activities were observed on the monitored endpoints.

Phase 3 evaluated whether the newly developed detection rules worked against realistic security telemetry. Atomic Red Team was used to simulate the targeted techniques on the lab endpoints and generate actual endpoint activity. The resulting telemetry was analyzed in Wazuh to determine whether each rule fired as expected. Several rules required tuning or improvements to telemetry collection before they generated the expected alerts. A significant finding was that Sysmon Operational logs were not initially being forwarded to Wazuh, which prevented some process-based detections from receiving the required telemetry. After correcting the log collection configuration and tuning affected rules, all five detection rules were successfully validated during simulation.

The validation process demonstrated the importance of distinguishing between detection logic and telemetry availability. A detection rule cannot generate an alert when the relevant event is not being collected by the SIEM. The project therefore involved both detection-rule tuning and log-source configuration improvements.

The completed validation achieved the Phase 3 checkpoint of at least four successful detections, with all five targeted rules ultimately firing successfully. The project also recorded the original and tuned configurations for rules that required adjustment, together with evidence of the resulting alerts.

Following detection validation, Phase 4 introduced a lightweight alert-triage workflow. A triage playbook was developed to define severity levels, escalation criteria, and alert disposition procedures. The workflow was then practiced using telemetry generated within the lab, including a successful SSH authentication scenario and a benign Windows system event. These exercises demonstrated how an analyst can correlate related events, assess severity, determine whether activity is malicious or legitimate, and decide whether an alert should be escalated or closed.

Overall, the project progressed from log-source validation, through detection engineering and simulated attack validation, to practical alert triage. The work demonstrated an end-to-end SOC monitoring workflow in which telemetry is collected, detection rules identify security-relevant activity, alerts are validated through simulation, and analysts apply a structured process to investigate and disposition those alerts.

Scope and Objectives

Scope

This project covers the validation, enhancement, testing, and operational use of a Security Operations Center (SOC) monitoring environment. The project is structured into four phases, with each phase building on the activities and outcomes of the previous phase.

Phase 1 — Log Source Validation and Gap Assessment

This phase focuses on validating the existing SOC lab environment and confirming that the required security log sources remain active and are successfully forwarding telemetry to Wazuh.

The phase covers:

- Windows Sysmon logs
- Windows Security logs
- Linux system and authentication logs
- Firewall logs
- Wazuh agent and manager communication
- Existing monitoring dashboards
- Identification and remediation of gaps carried over from the previous SOC lab

Phase 2 — Detection Engineering

This phase focuses on developing and implementing five Wazuh detection rules aligned with selected MITRE ATT&CK techniques.

The targeted techniques are:

- T1059.001 — PowerShell
- T1053.005 — Scheduled Task/Job: Scheduled Task
- T1021.004 — Remote Services: SSH
- T1087.001 — Account Discovery: Local Account
- T1057 — Process Discovery

Phase 3 — Detection Validation and Tuning

This phase focuses on validating the effectiveness of the detection rules by generating realistic security telemetry using Atomic Red Team simulations.

The phase includes:

- Simulating the selected MITRE ATT&CK techniques
- Confirming whether each detection rule generates the expected alert
- Identifying detections that fail to trigger
- Investigating telemetry and rule configuration gaps
- Tuning affected rules where necessary
- Re-running simulations after tuning
- Recording before-and-after results
- Confirming successful detection coverage

Phase 4 — Triage Workflow

This phase focuses on developing and practicing a lightweight SOC alert-triage workflow.

The phase includes:

- Defining Low, Medium, High, and Critical severity tiers
- Establishing escalation and closure criteria
- Correlating related security events
- Investigating generated alerts
- Determining alert disposition
- Documenting the reasoning behind each disposition
- Practicing the workflow using alerts generated within the laboratory environment

Objectives

The overall objective of the project is to demonstrate an end-to-end SOC monitoring capability that can collect security telemetry, detect adversary behaviors, validate detection effectiveness, and provide a structured process for analyzing and responding to alerts.

The project objectives are to:

Phase 1: Validate existing log sources and identify and remediate telemetry gaps within the SOC environment.

Phase 2: Develop five MITRE ATT&CK-aligned Wazuh detection rules for selected adversary techniques.

Phase 3: Use Atomic Red Team to simulate the targeted techniques, validate detection effectiveness, and tune detections where required.

Phase 4: Develop and apply a repeatable triage workflow for assessing, prioritizing, escalating, and closing security alerts.

The project ultimately aims to demonstrate the complete SOC monitoring lifecycle:

Log Collection → Detection Engineering → Simulation & Validation → Alert Triage → Disposition → Documentation

Methodology:

Phase 1 — Log Source Validation and Environment Verification

1.1 Objective

The objective of Phase 1 was to verify that the major log sources used in the previous SOC/SIEM laboratory environment were still operational and capable of generating security telemetry. The phase also focused on identifying gaps in log visibility and troubleshooting any configuration issues that prevented expected telemetry from reaching the Wazuh environment.

The primary log sources reviewed during this phase were:

- Windows Sysmon
- Windows Security logs
- Linux system and authentication logs

- pfSense firewall logs
- Wazuh agent and manager communication

The validation was performed before proceeding to detection engineering so that subsequent detection testing would be based on known and observable telemetry.

1.2 Windows Endpoint Log Validation

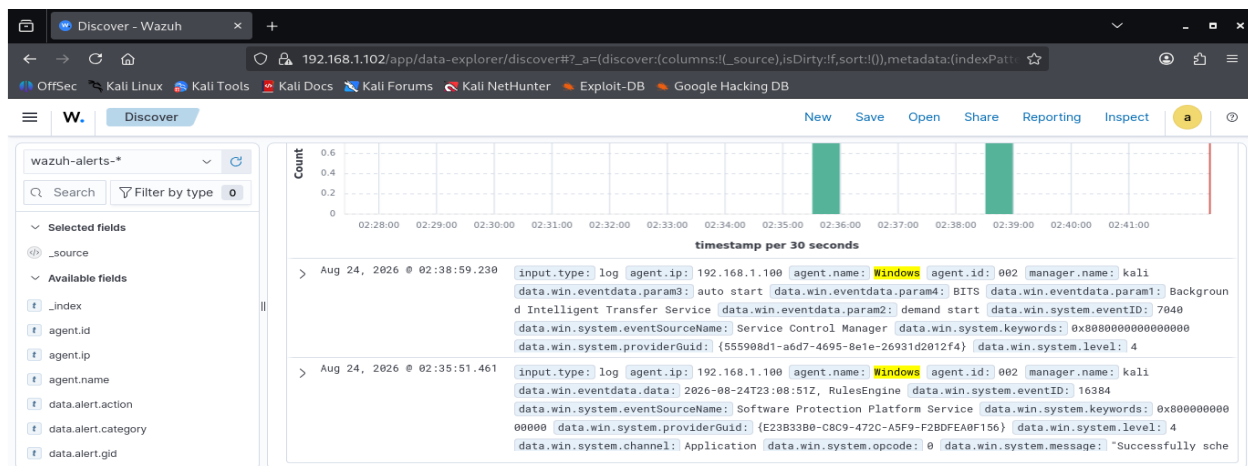
The Windows 11 endpoint was first checked to confirm that the Wazuh agent was operational and communicating with the Wazuh manager.

Windows event sources were then reviewed to confirm that endpoint activity was being generated. Particular attention was given to:

- Windows Security events
- Sysmon Operational events
- Process creation events
- Authentication-related events

The purpose of this check was to confirm that the endpoint was generating the telemetry required for later detection rules.

Relevant events were subsequently checked within Wazuh to determine whether the generated Windows telemetry was available to the SIEM.

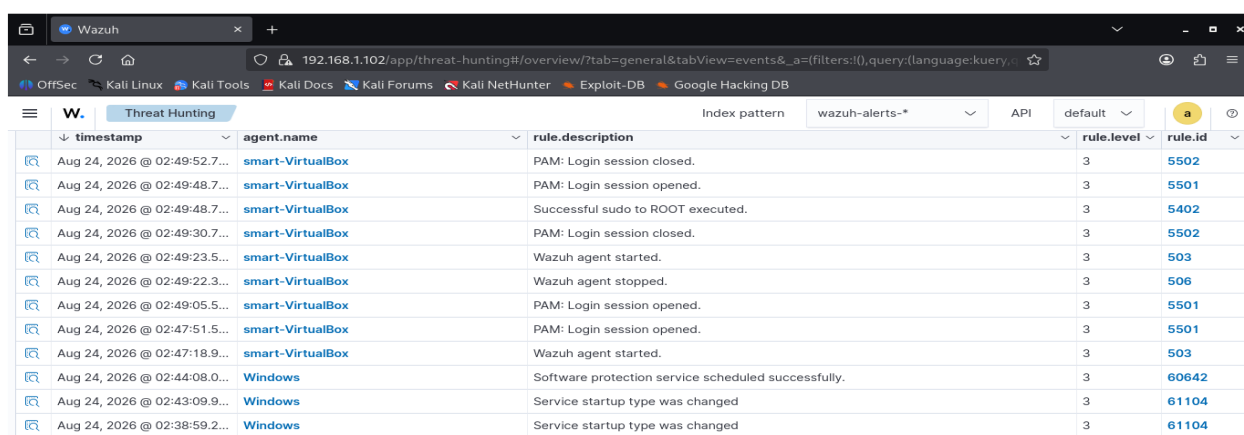


1.3 Linux Log Validation

The Linux endpoint was also reviewed to confirm that system and authentication logs were being generated and available for monitoring.

Authentication activity was particularly important because later detection testing involved SSH authentication events. The Linux logs were therefore examined for SSH-related activity, including authentication failures and successful authentication events.

The availability of these events was later confirmed during the SSH detection simulation and triage exercise.



The screenshot shows the Wazuh Threat Hunting dashboard. The browser address bar indicates the URL: 192.168.1.102/app/threat-hunting#/overview?tab=general&tabView=events&a=(filters:[](),query:(language:kuery...)). The dashboard has a navigation bar with 'W.' and 'Threat Hunting' tabs. Below the navigation bar, there are filters for 'Index pattern' (wazuh-alerts-*), 'API' (default), and 'rule.level' (3). The main table displays a list of events with columns: timestamp, agent.name, rule.description, rule.level, and rule.id. The events are sorted by timestamp, showing a sequence of login and agent status changes for 'smart-VirtualBox' and 'Windows' agents.

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 24, 2026 @ 02:49:52.7...	smart-VirtualBox	PAM: Login session closed.	3	5502
Aug 24, 2026 @ 02:49:48.7...	smart-VirtualBox	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 02:49:48.7...	smart-VirtualBox	Successful sudo to ROOT executed.	3	5402
Aug 24, 2026 @ 02:49:30.7...	smart-VirtualBox	PAM: Login session closed.	3	5502
Aug 24, 2026 @ 02:49:23.5...	smart-VirtualBox	Wazuh agent started.	3	503
Aug 24, 2026 @ 02:49:22.3...	smart-VirtualBox	Wazuh agent stopped.	3	506
Aug 24, 2026 @ 02:49:05.5...	smart-VirtualBox	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 02:47:51.5...	smart-VirtualBox	PAM: Login session opened.	3	5501
Aug 24, 2026 @ 02:47:18.9...	smart-VirtualBox	Wazuh agent started.	3	503
Aug 24, 2026 @ 02:44:08.0...	Windows	Software protection service scheduled successfully.	3	60642
Aug 24, 2026 @ 02:43:09.9...	Windows	Service startup type was changed	3	61104
Aug 24, 2026 @ 02:38:59.2...	Windows	Service startup type was changed	3	61104

1.4 pfSense Log Transmission Troubleshooting

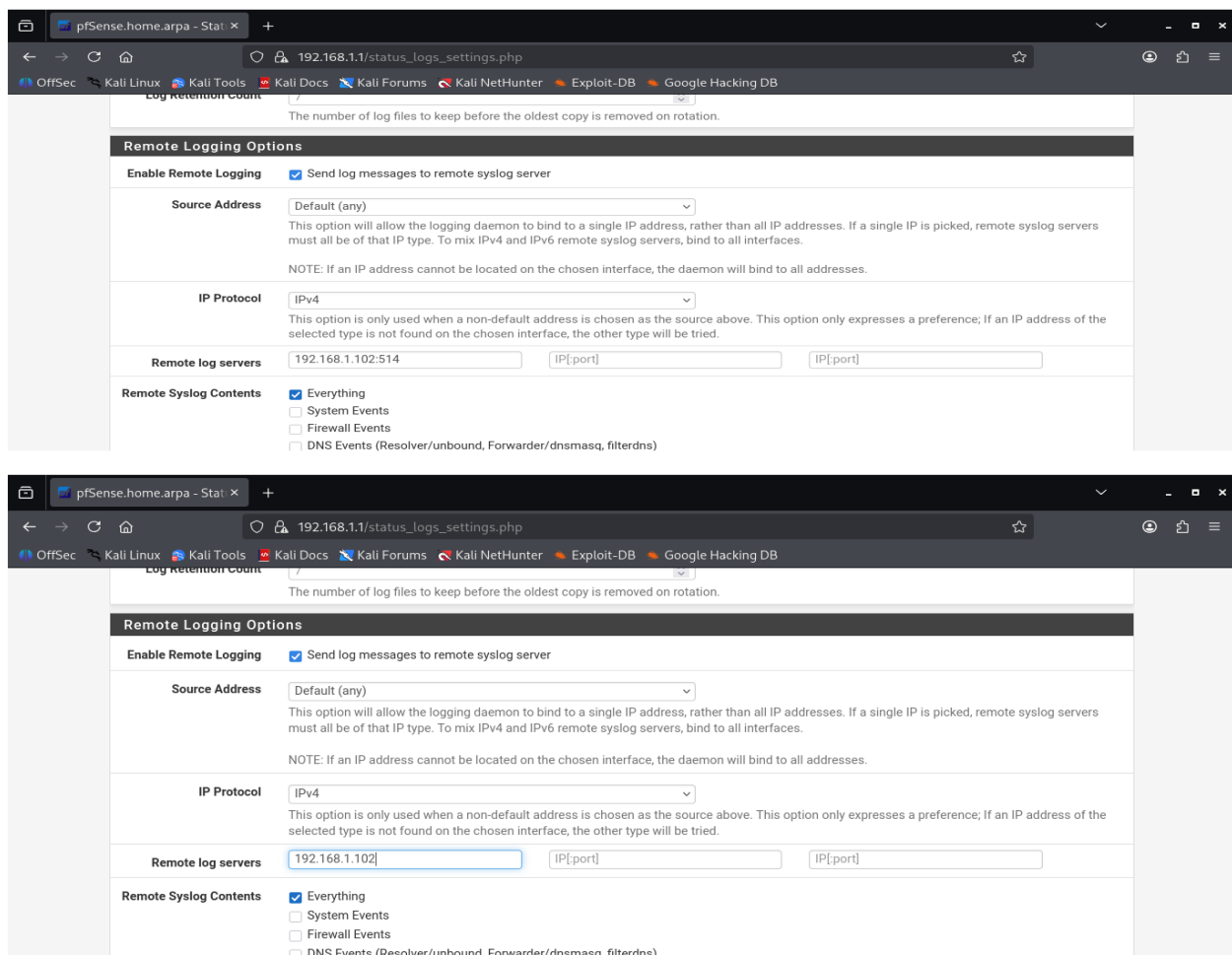
A logging gap was identified with the pfSense firewall. Although pfSense was configured to send logs to the Wazuh environment, the expected pfSense logs were not appearing in the Wazuh dashboard.

The pfSense remote logging configuration was therefore investigated.

Within the pfSense dashboard, the remote syslog configuration contained a destination IP address together with a port number. This configuration was tested and investigated because the expected log traffic was not initially visible at the Wazuh manager.

The configuration was modified by removing the port number from the destination field and leaving the Wazuh manager's IP address as the destination.

After applying the configuration change, pfSense began transmitting log traffic toward the Wazuh manager.



1.5 Verification Using tcpdump

After modifying the pfSense remote logging configuration, the Wazuh manager was monitored directly from the terminal using "tcpdump".

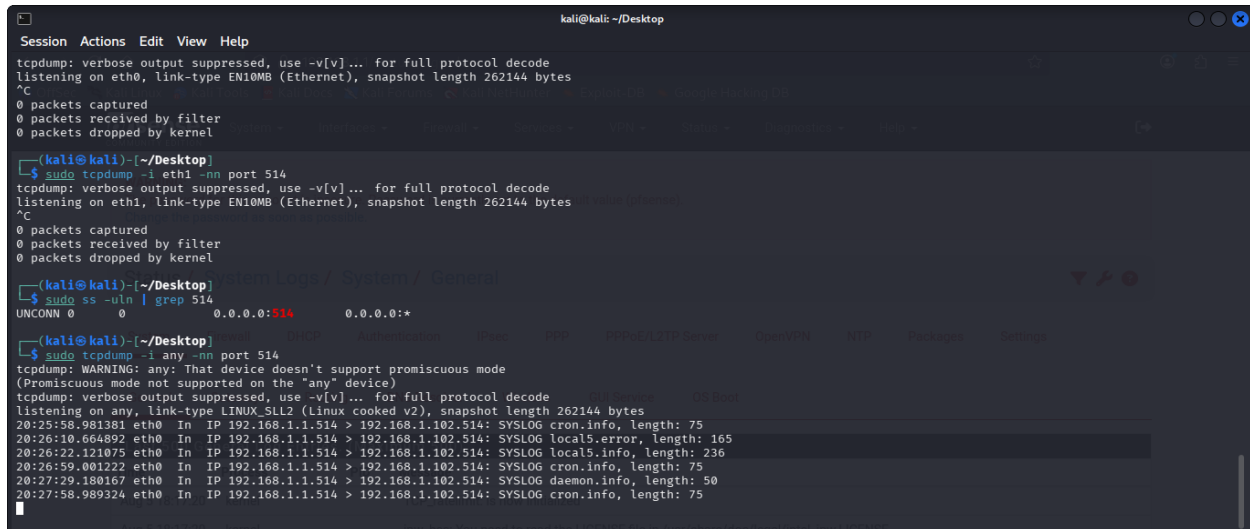
The purpose of using "tcpdump" was to determine whether pfSense log traffic was physically reaching the Wazuh manager, independently of whether the Wazuh dashboard displayed the events.

The packet capture confirmed that traffic from the pfSense device was reaching the Wazuh manager.

This established an important distinction between log transmission and log onboarding.

The troubleshooting successfully demonstrated that pfSense was transmitting the log traffic to the manager. However, the pfSense logs were still not onboarded into the Wazuh dashboard/indexed alert workflow. Therefore, pfSense was not considered a fully integrated Wazuh log source at the end of this phase.

The remaining issue was documented as a telemetry onboarding gap rather than being reported as a completed integration.



```
kali@kali: ~/Desktop
Session Actions Edit View Help
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel

(kali@kali)~$ sudo tcpdump -i eth1 -nn port 514
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel

(kali@kali)~$ sudo ss -tln | grep 514
UNCONN 0 0 0.0.0.0:514 0.0.0.0:*

(kali@kali)~$ sudo tcpdump -i any -nn port 514
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
20:25:58.981381 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG cron.info, length: 75
20:26:10.664892 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG local5.error, length: 165
20:26:22.121075 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG local5.info, length: 236
20:26:59.001222 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG cron.info, length: 75
20:27:29.188167 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG daemon.info, length: 50
20:27:58.989324 eth0 In IP 192.168.1.1.514 > 192.168.1.102.514: SYSLOG cron.info, length: 75
```

1.6 Phase 1 Validation Process

After reviewing the individual log sources, security activity was generated within the laboratory environment to confirm that the available telemetry could be observed in Wazuh.

The generated events were used to verify that the environment remained capable of supporting the detection engineering activities planned for the next phase.

The validation process therefore consisted of:

1. Checking the Windows endpoint and Wazuh agent.
2. Verifying Windows Security and Sysmon event generation.
3. Checking Linux system and authentication telemetry.
4. Reviewing the pfSense remote logging configuration.
5. Identifying the pfSense log transmission problem.
6. Correcting the pfSense destination configuration by removing the port number.
7. Monitoring the Wazuh manager with "tcpdump".
8. Confirming that pfSense log traffic reached the manager.
9. Confirming that pfSense traffic was still not fully onboarded into the Wazuh dashboard.

10. Recording the remaining pfSense onboarding limitation for the project.

1.7 Phase 1 Outcome

Phase 1 established the operational state of the existing SOC monitoring environment before detection development began.

Windows and Linux telemetry required for subsequent detection activities was available for monitoring. A pfSense logging issue was also investigated and corrected at the transmission level. The use of "tcpdump" confirmed that pfSense log traffic reached the Wazuh manager after the configuration change.

However, the investigation also established that receiving pfSense traffic at the Wazuh manager did not result in the logs being onboarded into the Wazuh dashboard. This distinction was recorded as an outstanding limitation of the environment.

The validated telemetry environment was then used as the foundation for Phase 2, where the project moved from log-source validation into detection engineering.

Phase 2 — Detection Engineering

2.1 Objective

The objective of Phase 2 was to develop and implement custom Wazuh detection rules for selected adversary behaviors identified through the MITRE ATT&CK framework.

The detections were designed based on the telemetry available from the validated Windows and Linux endpoints. Each rule was configured to identify a specific command, process, authentication activity, or system behavior associated with the selected MITRE ATT&CK technique.

The detection rules were created within the Wazuh custom rules configuration and assigned appropriate rule IDs, severity levels, descriptions, and MITRE ATT&CK mappings.

2.2 Detection Rule Development Process

The detection engineering process began by selecting five MITRE ATT&CK techniques that represented relevant discovery, execution, persistence, and remote-access behaviors within the laboratory environment.

For each selected technique, the corresponding endpoint activity was identified and the available Wazuh event fields were reviewed to determine which fields could be used to detect the behavior.

The general rule-development process was:

1. Select the MITRE ATT&CK technique.
2. Identify the activity associated with the technique.
3. Determine the expected telemetry generated by the activity.
4. Identify the relevant Wazuh event fields.
5. Create a custom Wazuh rule using the appropriate event field and detection condition.
6. Assign a suitable alert level and descriptive message.
7. Map the rule to the corresponding MITRE ATT&CK technique.
8. Save the rule within the Wazuh custom rules configuration.
9. Validate that Wazuh accepted the rule configuration.
10. Prepare the rule for testing during Phase 3.

The five detection areas implemented during this phase are described below.

2.3 PowerShell Detection

The first detection was developed to identify PowerShell execution on the Windows endpoint.

The rule was designed to inspect Windows process-execution telemetry and identify PowerShell activity based on the command/process information contained within the event.

The detection was mapped to:

MITRE ATT&CK: T1059.001 — Command and Scripting Interpreter: PowerShell

The rule was assigned a custom Wazuh rule ID and an appropriate severity level to ensure that detected PowerShell activity generated a visible security alert.

2.4 Scheduled Task Detection

The second detection was developed to identify the creation or execution of scheduled tasks on the Windows endpoint.

Scheduled tasks can be used by adversaries to establish persistence or execute commands at predetermined times. The detection therefore focused on identifying relevant scheduled-task activity within Windows event telemetry.

The detection was mapped to:

MITRE ATT&CK: T1053.005 — Scheduled Task/Job: Scheduled Task

The rule was configured to generate a Wazuh alert when the relevant scheduled-task event was observed.

2.5 SSH Remote Service Detection

The third detection was developed to identify SSH remote-service activity.

Linux authentication and SSH telemetry were used as the basis for this detection. The rule was designed to identify SSH activity and associate the observed behavior with the MITRE ATT&CK remote-services technique.

The detection was mapped to:

MITRE ATT&CK: T1021.004 — Remote Services: SSH

The rule was configured with an appropriate severity level and descriptive alert message to provide visibility into SSH remote-service activity.

2.6 Local Account Discovery Detection

The fourth detection was developed to identify attempts to enumerate local accounts on the Windows endpoint.

The detection targeted commands and activity associated with account discovery, including Windows commands capable of displaying local users, groups, and account-related information.

The detection was mapped to:

MITRE ATT&CK: T1087.001 — Account Discovery: Local Account

The rule was designed to identify the relevant command execution telemetry and generate an alert when local account enumeration activity was detected.

The detection was subsequently tested using Atomic Red Team account-discovery simulations during Phase 3.

2.7 Process Discovery Detection

The fifth detection was developed to identify process discovery activity on the Windows endpoint.

The detection focused on process-enumeration activity, including the use of Windows commands such as "tasklist" to identify running processes.

The detection was mapped to:

MITRE ATT&CK: T1057 — Process Discovery

The rule was configured to inspect the relevant Windows process-execution telemetry and generate an alert when the targeted process-discovery behavior was observed.

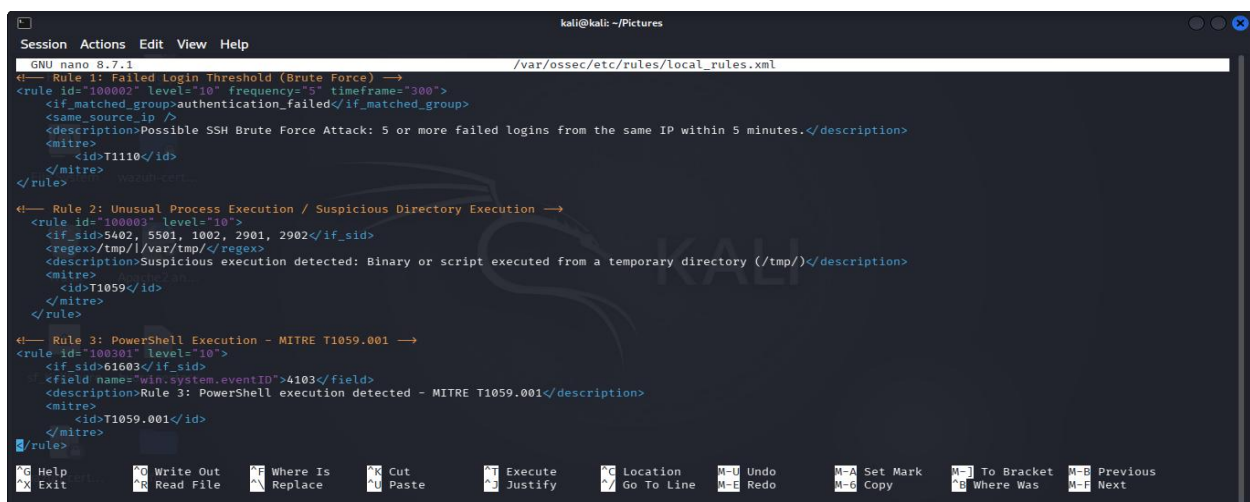
2.8 Rule Configuration and Validation

After creating the five custom rules, the Wazuh rule configuration was reviewed to ensure that the syntax and rule relationships were correctly configured.

The custom rules were loaded into the Wazuh detection engine, after which the environment was prepared for controlled simulation testing.

At this stage, the purpose was not yet to determine whether each detection successfully identified adversary behavior. Instead, Phase 2 established the detection logic and prepared the rules for controlled validation in Phase 3.

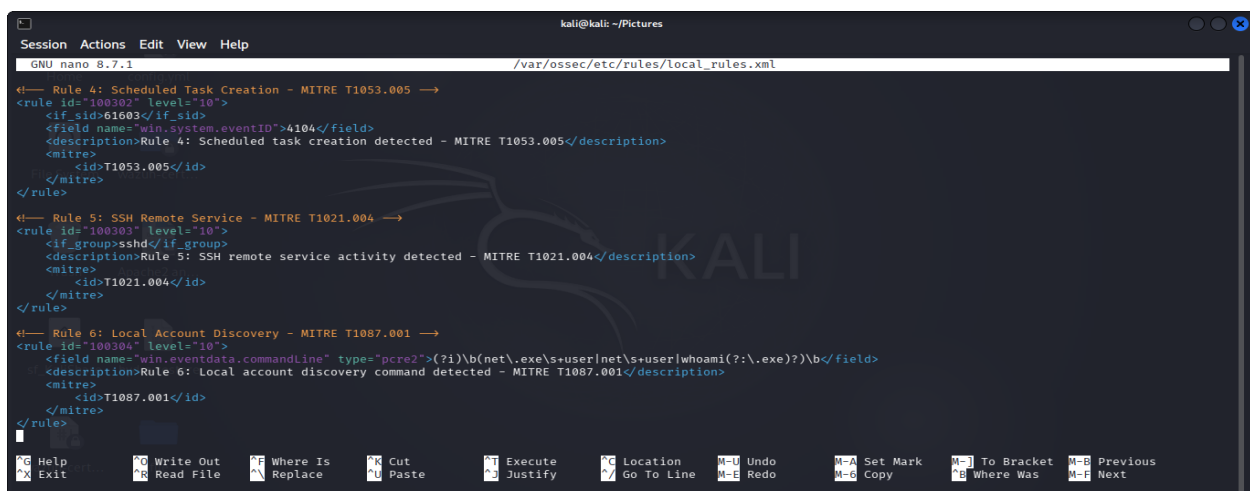
The raw XML configurations are retained as technical evidence through screenshots of the Wazuh rules configuration. The detailed rule syntax is not reproduced in this section because the focus of Phase 2 is the detection-engineering process, while Phase 3 documents the specific tuning performed when detections did not initially produce the expected alerts.



```
GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml
<!-- Rule 1: Failed Login Threshold (Brute Force) -->
<rule id="100002" level="10" frequency="5" timeframe="300">
  <if_matched_group>authentication_failed</if_matched_group>
  <same_source_ip />
  <description>Possible SSH Brute Force Attack: 5 or more failed logins from the same IP within 5 minutes.</description>
  <mitre>
    <id>T1110</id>
  </mitre>
</rule>

<!-- Rule 2: Unusual Process Execution / Suspicious Directory Execution -->
<rule id="100003" level="10">
  <if_sid>5402, 5501, 1002, 2901, 2902</if_sid>
  <regex>/tmp/|/var/tmp/</regex>
  <description>Suspicious execution detected: Binary or script executed from a temporary directory (/tmp/)</description>
  <mitre>
    <id>T1059</id>
  </mitre>
</rule>

<!-- Rule 3: PowerShell Execution - MITRE T1059.001 -->
<rule id="100301" level="10">
  <if_sid>61603</if_sid>
  <field name="win.system.eventID">4103</field>
  <description>Rule 3: PowerShell execution detected - MITRE T1059.001</description>
  <mitre>
    <id>T1059.001</id>
  </mitre>
</rule>
```



```
GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml

<!-- Rule 4: Scheduled Task Creation - MITRE T1053.005 -->
<rule id="100302" level="10">
  <if_sid>61603</if_sid>
  <field name="win.system.eventID">4104</field>
  <description>Rule 4: Scheduled task creation detected - MITRE T1053.005</description>
  <mitre>
    <id>T1053.005</id>
  </mitre>
</rule>

<!-- Rule 5: SSH Remote Service - MITRE T1021.004 -->
<rule id="100303" level="10">
  <if_group>ssh</if_group>
  <description>Rule 5: SSH remote service activity detected - MITRE T1021.004</description>
  <mitre>
    <id>T1021.004</id>
  </mitre>
</rule>

<!-- Rule 6: Local Account Discovery - MITRE T1087.001 -->
<rule id="100304" level="10">
  <field name="win.eventdata.commandLine" type="pcr2?">(?!)\b(net\.exe\s+user|net\s+user|whoami(?:\.exe)?)\b</field>
  <description>Rule 6: Local account discovery command detected - MITRE T1087.001</description>
  <mitre>
    <id>T1087.001</id>
  </mitre>
</rule>
```



```
Session Actions Edit View Help
GNU nano 2.7.1 /var/ossec/etc/rules/local_rules.xml

<!-- Rule 7: Process Discovery - MITRE T1057 -->
<rule id="100305" level="10">
  <if_sid>61003</if_sid>
  <field name="win.eventdata.commandLine">(?i)tasklist(\\.exe)?</field>
  <description>Rule 7: Process discovery detected - MITRE T1057</description>
  <mitre>
    <id>T1057</id>
  </mitre>
</rule>
</group>

<group name="syscheck">
  <rule id="100200" level="7">
    <if_sid>550</if_sid>
    <description>File modified in /root</description>
  </rule>

  <rule id="100201" level="7">
    <if_sid>554</if_sid>
    <description>File added to /root</description>
  </rule>
</group>

<group name="pfsense,firewall">
  <rule id="100010" level="0">
    <program_name>filterlog</program_name>
  </rule>
</group>
```

2.9 Phase 2 Outcome

At the completion of Phase 2, five custom Wazuh detection rules had been developed and mapped to their respective MITRE ATT&CK techniques.

The rules established the detection logic required for the next phase, where Atomic Red Team simulations were used to generate the targeted behaviors and determine whether the rules successfully detected the resulting telemetry.

The effectiveness of the rules, including any detection failures, telemetry issues, and tuning requirements, was therefore evaluated in Phase 3.

3 Phase 3 — Detection Validation and Tuning

3.1 Objective

The objective of Phase 3 was to validate the five custom Wazuh detection rules developed during Phase 2 by generating controlled adversary activity against the laboratory endpoints.

Atomic Red Team was used to execute tests corresponding to the selected MITRE ATT&CK techniques. The resulting endpoint telemetry was then monitored in Wazuh to determine whether the appropriate detection rules generated alerts.

Where a detection did not initially generate the expected alert, the underlying telemetry and detection logic were investigated. Any required correction was then applied and the Atomic Red Team test was repeated to confirm successful detection.

3.2 Atomic Red Team Preparation

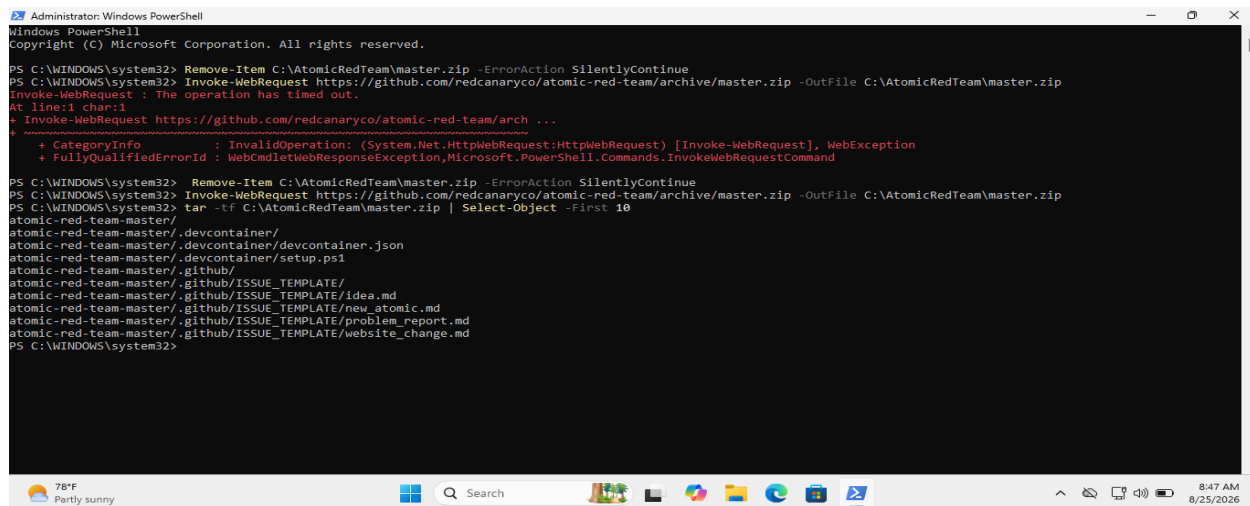
The first stage of Phase 3 involved preparing Atomic Red Team for controlled attack simulation.

The required Atomic Red Team testing environment was established and the appropriate atomic tests were identified according to the MITRE ATT&CK techniques targeted by the five custom Wazuh rules.

Each test was reviewed to determine:

- The MITRE ATT&CK technique being simulated.
- The command or activity that would be executed.
- The endpoint on which the test would run.
- The telemetry expected to be generated.
- The Wazuh rule expected to detect the activity.

The tests were executed individually so that each generated event could be associated with its corresponding detection rule.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> Remove-Item C:\AtomicRedTeam\master.zip -ErrorAction SilentlyContinue
PS C:\WINDOWS\system32> Invoke-WebRequest https://github.com/redcanaryco/atomic-red-team/archive/master.zip -OutFile C:\AtomicRedTeam\master.zip
Invoke-WebRequest : The operation has timed out.
At line:1 char:1
+ Invoke-WebRequest https://github.com/redcanaryco/atomic-red-team/arch ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\WINDOWS\system32> Remove-Item C:\AtomicRedTeam\master.zip -ErrorAction SilentlyContinue
PS C:\WINDOWS\system32> Invoke-WebRequest https://github.com/redcanaryco/atomic-red-team/archive/master.zip -OutFile C:\AtomicRedTeam\master.zip
PS C:\WINDOWS\system32> tar -tf C:\AtomicRedTeam\master.zip | Select-Object -First 10
atomic-red-team-master/
atomic-red-team-master/.devcontainer/devcontainer.json
atomic-red-team-master/.devcontainer/setup.ps1
atomic-red-team-master/.github/
atomic-red-team-master/.github/ISSUE_TEMPLATE/
atomic-red-team-master/.github/ISSUE_TEMPLATE/idea.md
atomic-red-team-master/.github/ISSUE_TEMPLATE/new_atomic.md
atomic-red-team-master/.github/ISSUE_TEMPLATE/problem_report.md
atomic-red-team-master/.github/ISSUE_TEMPLATE/website_change.md
PS C:\WINDOWS\system32>
```

3.3 Detection Testing Procedure

Each detection was validated by executing its corresponding Atomic Red Team test and monitoring the Wazuh environment for the expected alert.

The testing procedure consisted of:

1. Selecting the MITRE ATT&CK technique to be tested.
2. Selecting the corresponding Atomic Red Team test.
3. Reviewing the test command and execution requirements.
4. Executing the test against the appropriate endpoint.
5. Monitoring the endpoint for the generated activity.
6. Searching Wazuh for the resulting telemetry.
7. Checking whether the expected custom detection rule fired.
8. Reviewing the generated alert and its event fields.
9. Recording the result.

10. Investigating any detection that did not initially produce the expected alert.
11. Applying the required correction or tuning.
12. Repeating the test.
13. Confirming the final detection result.

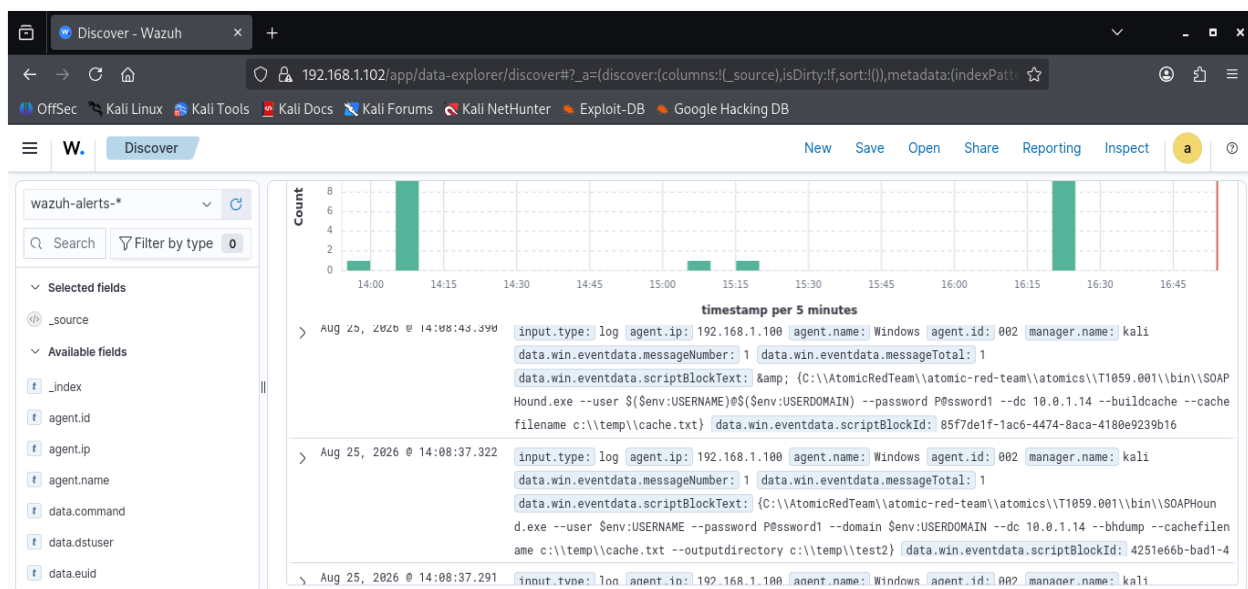
3.4 PowerShell Detection Validation

The PowerShell detection was tested using an Atomic Red Team simulation corresponding to MITRE ATT&CK T1059.001 — Command and Scripting Interpreter: PowerShell.

The simulation generated PowerShell activity on the Windows endpoint. The resulting Windows telemetry was monitored in Wazuh and the corresponding custom detection rule was evaluated.

The expected Wazuh alert was successfully generated, confirming that the PowerShell detection was functioning as intended.

No rule tuning was required for this detection.



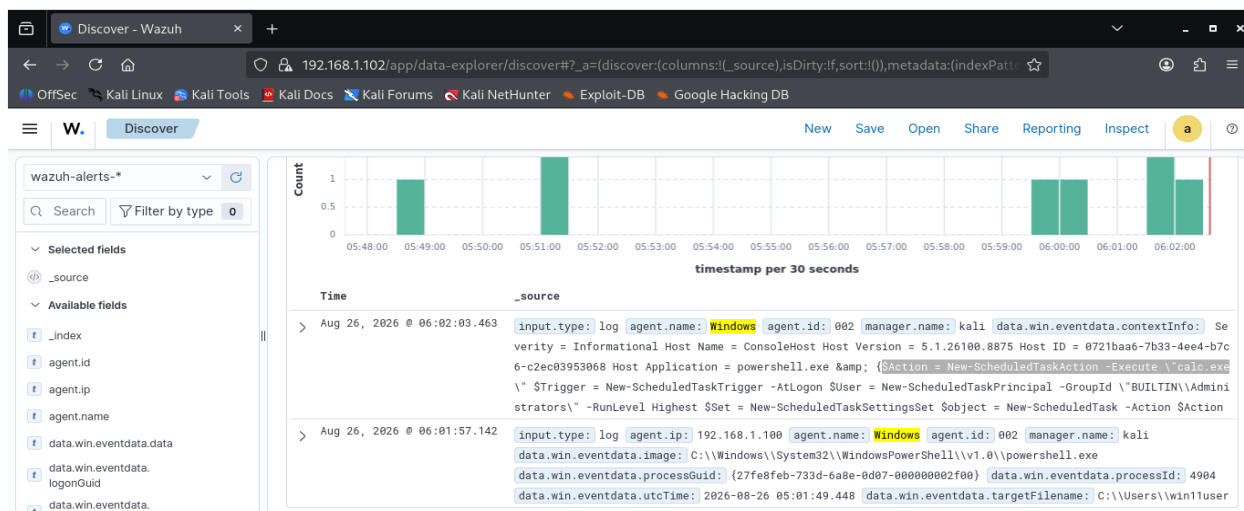
3.5 Scheduled Task Detection Validation

The Scheduled Task detection was tested using an Atomic Red Team simulation corresponding to MITRE ATT&CK T1053.005 — Scheduled Task/Job: Scheduled Task.

The simulation generated scheduled-task activity on the Windows endpoint. The resulting telemetry was monitored within Wazuh to determine whether the custom detection rule identified the activity.

The expected detection alert was successfully generated.

No rule tuning was required for this detection.



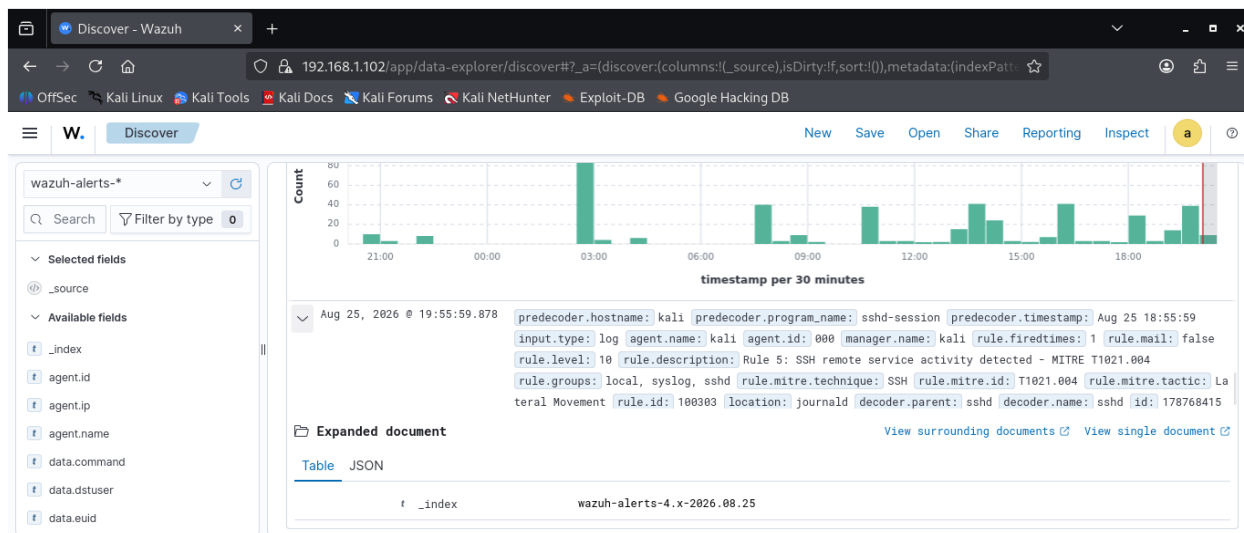
3.6 SSH Remote Service Detection Validation

The SSH detection was tested using SSH-related activity corresponding to MITRE ATT&CK T1021.004 — Remote Services: SSH.

The generated Linux authentication and SSH telemetry was monitored within Wazuh.

The detection successfully generated the expected alert. The resulting SSH alerts were later used during Phase 4 for practical alert triage and correlation.

No rule tuning was required for this detection.



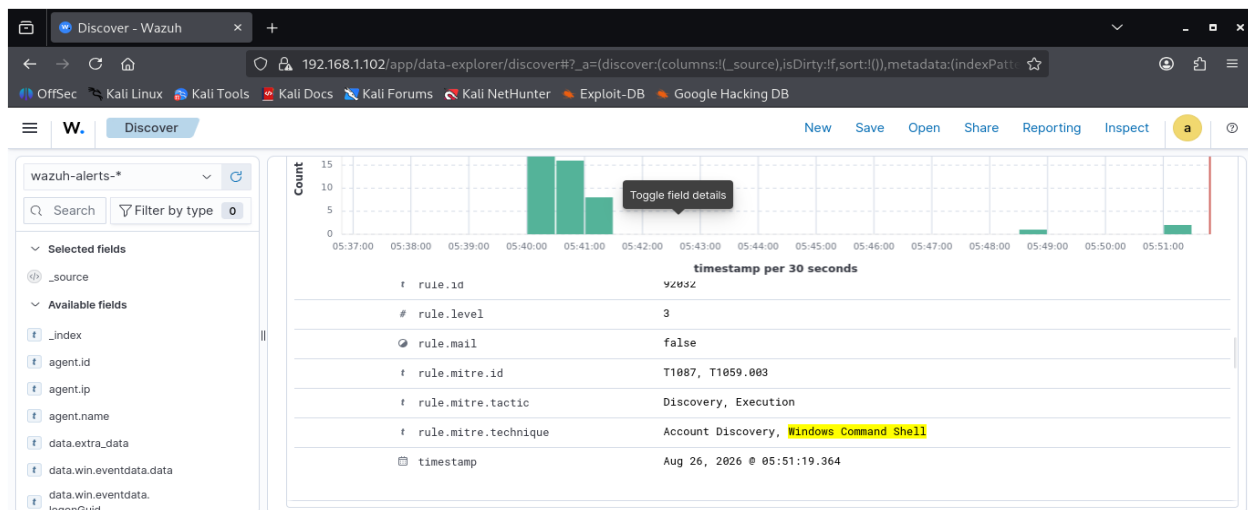
3.7 Local Account Discovery Detection Validation

The Local Account Discovery detection was tested using an Atomic Red Team simulation corresponding to MITRE ATT&CK T1087.001 — Account Discovery: Local Account.

The test generated account-discovery activity on the Windows endpoint. The resulting telemetry was reviewed in Wazuh to confirm that the custom detection rule identified the simulated behavior.

The expected Wazuh alert was successfully generated.

No rule tuning was required for this detection.



3.8 Process Discovery Detection Validation

The Process Discovery detection was tested using the Atomic Red Team test Process Discovery - tasklist, corresponding to MITRE ATT&CK T1057 — Process Discovery.

The Atomic test executed the following command:

tasklist

The purpose of the simulation was to generate process-enumeration activity that could be identified through Windows process telemetry.

The initial execution did not produce the expected custom Wazuh alert.

The detection was therefore investigated to determine whether the issue was caused by the rule logic or by missing telemetry.

3.9 Sysmon Telemetry Investigation

During the investigation, it was identified that the relevant Sysmon Operational telemetry was not initially being forwarded to Wazuh.

This was significant because the Process Discovery detection depended on Windows process-execution telemetry.

The Windows endpoint was generating the relevant activity, but the required Sysmon events were not available to the Wazuh detection engine. Therefore, the absence of the expected alert could not initially be attributed solely to the detection rule.

The Sysmon telemetry forwarding configuration was investigated and corrected.

After the correction, the relevant Sysmon events became visible within Wazuh.

The Process Discovery Atomic Red Team test was then executed again.

3.10 Process Discovery Rule Tuning

After the required Sysmon telemetry became available, the Process Discovery detection condition was reviewed against the actual Windows event fields received by Wazuh.

The original detection was adjusted so that it matched the command-line field containing the "tasklist" activity.

Original Detection Condition

```
<field name="win.eventdata.commandLine">{?i}tasklist</field>
```

Tuned Detection Condition

```
<field name="win.eventdata.commandLine" type="pcre2">{?i}tasklist</field>
```

The tuning added the "pcre2" matching type to the command-line field condition while retaining the case-insensitive "tasklist" pattern.

The purpose of the change was to ensure that the detection condition correctly evaluated the command-line telemetry being received from the Windows endpoint.

Result Before Tuning

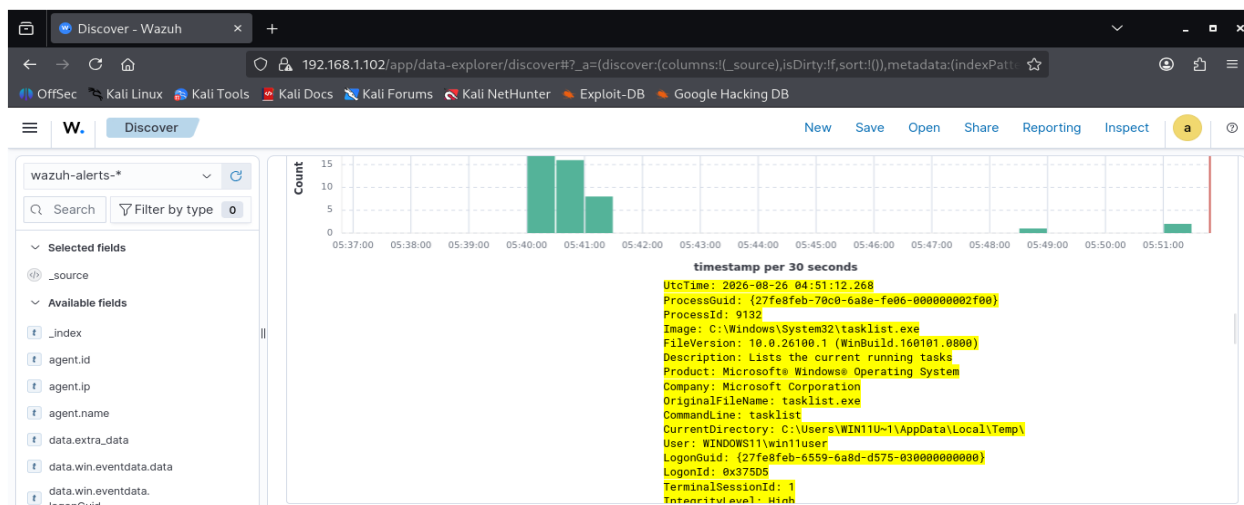
The Atomic Red Team Process Discovery test was executed, but the expected custom Wazuh alert was not generated.

Result After Tuning

After the telemetry forwarding issue was corrected and the detection condition was tuned, the same Atomic Red Team test was executed again.

The resulting Windows telemetry was successfully received by Wazuh and the custom Process Discovery rule generated the expected alert.

The event showed the relevant process information, including "tasklist.exe", confirming successful detection of the simulated Process Discovery activity.



3.11 Final Detection Validation

Following the testing and troubleshooting process, all five targeted detection rules successfully generated the expected alerts during simulation.

The final validation status was:

Detection	MITRE ATT&CK	Initial Result	Tuning Required	Final Result
PowerShell	T1059.001	Successful	No	Successful
Scheduled Task	T1053.005	Successful	No	Successful
SSH Remote Svc	T1021.004	Successful	No	Successful
Local Account Discovery	T1087.001	Successful	No	Successful
Process Discovery	T1057	No alert initially	Yes	Successful

The Process Discovery detection was the only detection that required rule tuning during the documented validation process. The remaining four detections generated the expected alerts without requiring rule modification.

3.12 Tuning and Troubleshooting Record

The Process Discovery test demonstrated that detection failures must be investigated from both the telemetry and detection-rule perspectives.

The investigation identified that Sysmon telemetry was not initially being forwarded to Wazuh. After the telemetry path was corrected, the Process Discovery detection condition was adjusted to correctly evaluate the command-line event field.

The validation was then repeated using the same Atomic Red Team test.

The final sequence was:

Atomic test → no expected alert → telemetry investigation → Sysmon forwarding correction → rule condition review → rule tuning → Atomic test repeated → Wazuh alert generated.

The original and tuned detection conditions are documented in this section, while the resulting Wazuh alert provides evidence that the corrected detection successfully identified the simulated behavior.

3.13 Phase 3 Outcome

Phase 3 successfully validated the five custom Wazuh detections through controlled Atomic Red Team simulations.

The testing demonstrated that the detection pipeline must be evaluated as a complete process, including attack simulation, endpoint telemetry generation, log forwarding, event visibility, detection logic, and final alert generation.

Four detections successfully generated alerts without requiring rule tuning. The Process Discovery detection initially failed to generate the expected alert and required investigation. The investigation identified a Sysmon telemetry forwarding issue, after which the detection condition was tuned and the simulation repeated successfully.

The final testing resulted in successful validation of all five targeted detections.

The alerts generated during this phase provided the practical security events used in Phase 4 to develop and apply the triage workflow.

Phase 4 — Triage Workflow

4.1 Triage Workflow Introduction

A triage playbook was developed based on the detection and investigation requirements of the SOC lab. The playbook was then applied to two alerts generated during the detection-validation phase: one confirmed true-positive alert and one false-positive alert.

The first alert involved SSH authentication activity and was investigated through correlation of multiple failed authentication events with a subsequent successful authentication from the same source IP address. Based on the correlated evidence, the alert was treated as a confirmed security event and escalated.

The second alert involved legitimate Windows system activity. The alert was investigated and correlated with surrounding system events, after which the activity was determined to be benign and the alert was closed as a false positive.

4.2 Triage Playbook

The following playbook was developed to provide a consistent procedure for handling security alerts within the SOC environment.

Step 1: Severity Triage Tiers

Low Severity

Alerts that are likely benign, expected, or have minimal security impact. The activity can be closed after basic validation confirms there is no indication of malicious behavior.

Example: An authorized administrator running "tasklist" during normal system administration.

Medium Severity

Alerts involving suspicious or unusual activity that require further investigation but do not currently indicate an active compromise or significant impact.

Example: Unusual PowerShell execution by a user that requires additional context to determine whether the activity is legitimate.

High Severity

Alerts with strong indicators of malicious activity or a possible compromise of an endpoint or account. These alerts require prompt investigation and may require escalation.

Example: Suspicious PowerShell execution combined with unusual user activity or other indicators of compromise.

Critical Severity

Alerts indicating confirmed or ongoing compromise, significant impact, or an immediate threat to critical systems, accounts, or data. These alerts require immediate escalation and response.

Example: Confirmed malicious activity involving credential theft, lateral movement, or data exfiltration.

Severity Assessment Factors

Severity is determined using the following factors:

- Confidence that the activity is malicious
- Potential impact of the activity
- Criticality of the affected asset
- Whether the activity is expected for the user or system
- Supporting evidence from correlated events
- Scope of the activity and whether multiple systems are affected

Step 2: Escalation Criteria

Close the Alert

An alert may be closed when investigation confirms that the activity is legitimate, authorized, or otherwise benign, and there are no additional indicators of compromise.

The analyst should document the evidence and reason for closing the alert.

Example: An administrator legitimately executes "tasklist" as part of routine system administration.

Continue Investigation

An alert should remain open for further investigation when the activity is suspicious but there is insufficient evidence to confirm malicious activity.

The analyst should collect additional context, correlate related events, and monitor for further suspicious activity.

Example: PowerShell execution is detected from an unusual user, but there is not enough evidence to determine whether the activity is malicious.

Escalate the Alert

An alert should be escalated when there is strong evidence of malicious activity, a potential compromise, or activity that requires additional investigation or response beyond the analyst's current level of responsibility.

Examples include:

- Multiple correlated security alerts indicating an attack
- Confirmed malicious execution
- Suspicious activity involving privileged accounts
- Evidence of lateral movement
- Evidence of credential theft or unauthorized access
- Multiple affected endpoints

Immediate Escalation

Critical alerts involving confirmed or ongoing compromise, significant impact, or an immediate threat to critical systems or sensitive data should be escalated immediately to the appropriate security or incident response team.

Escalation Information

When escalating an alert, the analyst should provide:

- Alert name and severity

- Affected endpoint(s)
- User/account involved
- Date and time of activity
- Relevant commands, processes, or network activity
- Correlated alerts or supporting evidence
- Initial assessment and reason for escalation
- Actions already taken

Triage Example — Successful SSH Authentication

Alert Summary

A high-severity SSH remote service alert was generated on the Kali endpoint. The alert was associated with MITRE ATT&CK technique T1021.004 — SSH, under the Lateral Movement tactic.

Initial Assessment

The alert was investigated by correlating the successful SSH authentication event with preceding SSH authentication-failure events.

Multiple failed authentication attempts were observed from the same source IP address, 192.168.1.100, against the Kali host. The events included repeated authentication failures associated with T1110.001 — Password Guessing.

A subsequent SSH event showed:

- Authentication: Accepted password
- Username: kali
- Source IP: 192.168.1.100
- Host/Agent: Kali
- Protocol: SSH
- MITRE ATT&CK: T1021.004 — SSH
- Tactic: Lateral Movement

Correlation and Analysis

The repeated authentication failures followed by a successful password authentication from the same source IP provided a strong correlation between the events.

The sequence observed was:

Multiple failed SSH authentication attempts → successful password authentication → SSH access established

In a real-world environment, this sequence would represent a strong indicator of potentially unauthorized access and possible lateral movement. The successful authentication following multiple failures increases the confidence that the account may have been compromised or that unauthorized credentials were successfully used.

No evidence of credential theft, data exfiltration, or activity involving multiple affected endpoints was identified during this limited triage.

Severity Assessment

Severity: Critical

The alert would be classified as Critical in a real-world environment because the correlated events indicate successful access following multiple authentication failures, representing a potential confirmed or ongoing unauthorized access event.

Disposition

Lab disposition: Benign / Authorized Simulation

The activity was intentionally generated as part of an authorized Atomic Red Team simulation within the lab environment. Therefore, the alert itself does not represent an actual compromise of the lab.

Real-world disposition: Escalate

If the same sequence occurred unexpectedly in a production environment, the alert should be escalated for further investigation and incident response.

Escalation Reasoning

The alert should be escalated because:

- Multiple SSH authentication failures were observed.
- The failures originated from the same source IP.
- A subsequent successful password authentication occurred from that same source IP.
- The successful authentication resulted in SSH access to the affected host.
- The correlated events provide stronger evidence than either a single failed login or a single successful login viewed independently.
- The activity is associated with Lateral Movement — T1021.004 (SSH).

The appropriate next step in a real-world environment would be to escalate the incident for deeper investigation, including verification of whether the account and source IP were authorized, examination of activity following the successful login, and assessment for additional indicators of compromise.

Triage Conclusion

The correlation of multiple failed SSH authentication attempts with a subsequent successful password authentication demonstrates why individual alerts should be evaluated in context rather than in isolation.

Although this event was an authorized simulation in the lab, the same telemetry pattern in an unexpected production environment would warrant immediate escalation.

Triage Example — Benign System Error

Alert Summary

A Windows system error alert was observed on the Windows 11 endpoint.

The event reported a failed Secure Boot update involving Option ROM CA 2023 (DB). The system reported that it could not find the specified file.

Alert details:

- Agent: Windows 11
- Agent IP: 192.168.1.100
- Computer: Windows 11
- Timestamp: August 26, 2026, 11:26:30
- Log source: Windows Channel
- Update type: Option ROM CA 2023 (DB)
- System message: Secure Boot update failed because the system could not find the specified file -
- Severity: Error

Initial Assessment

The alert initially appeared significant because it was classified by Windows as an Error. However, the event itself describes a system update failure rather than evidence of malicious activity.

The event was therefore investigated for related activity and additional security indicators.

Correlation

A nearby Windows event from the same endpoint was observed at approximately 11:26:37.

That event reported:

- Agent: Windows 11
- Agent IP: 192.168.1.100
- Computer: Windows 11
- Timestamp: August 26, 2026, 11:26:37

- Event: Software Protection successfully scheduled
- Severity: Information

The second event occurred shortly after the Secure Boot update error and originated from the same Windows endpoint. However, the events do not provide sufficient evidence to establish that they are directly related.

The Software Protection event is consistent with normal Windows system activity and does not provide an indicator of compromise.

Severity Assessment

Severity: Low

The event represents a Windows system/update error but does not contain evidence of malicious execution, unauthorized access, persistence, lateral movement, or other suspicious security activity.

Disposition

Benign / False Positive — Close

The alert is closed after triage because the available evidence indicates a legitimate Windows system/update issue rather than malicious activity.

Reasoning

The alert was classified as benign because:

- The event originated from the Windows operating system.
- The event describes a failed Secure Boot update rather than malicious execution.
- No suspicious process, account, network connection, or unauthorized access was identified from the available evidence.
- A separate informational Windows event from the same endpoint showed normal Software Protection activity.
- No additional indicators of compromise were identified during the limited triage.

The Windows Error severity indicates a system problem, but an operating-system error does not automatically represent a security incident.

Triage Conclusion

The alert was investigated and determined to be a benign system error rather than a security threat. The alert was therefore closed without escalation.

This demonstrates that security triage should distinguish between system errors and genuine security incidents, rather than escalating every event simply because the operating system labels it as an error.

Findings

The project successfully validated the SOC monitoring and detection workflow across the laboratory environment. The review confirmed that the available Windows, Sysmon, and Linux telemetry could be used to generate and investigate security events within Wazuh. The pfSense firewall logs were visible at the Wazuh manager during troubleshooting through network-level verification with TCPDump, although the pfSense logs were not successfully onboarded into the Wazuh dashboard.

Five custom detection rules were tested using controlled Atomic Red Team simulations, and all five ultimately generated the expected detections. Four rules functioned without requiring tuning, while the Process Discovery detection required troubleshooting and tuning. The investigation identified that the relevant Sysmon telemetry was not initially being forwarded to Wazuh. After correcting the telemetry forwarding issue and adjusting the detection condition, the Process Discovery simulation successfully generated the expected alert.

The project also demonstrated the importance of correlating multiple security events rather than evaluating individual alerts in isolation. During the SSH triage exercise, multiple authentication failures from the same source IP were correlated with a subsequent successful authentication, providing sufficient evidence for escalation. A separate Windows system alert was investigated and determined to be legitimate system activity, resulting in a false-positive disposition and closure.

Overall, the project demonstrated an end-to-end SOC workflow covering telemetry validation, detection engineering, adversary simulation, detection tuning, alert investigation, event correlation, severity assessment, and escalation or closure decisions.

Challenges and Gaps

Several technical challenges and gaps were encountered during the implementation and validation phases of the project.

The first challenge involved preparing Atomic Red Team for use in the laboratory. The initial attempt to obtain and extract the required Atomic Red Team components was unsuccessful, requiring an alternative approach. The repository was subsequently cloned and configured successfully, allowing the required simulations to be performed.

A second challenge occurred during detection validation when some expected telemetry was not initially available in Wazuh. Investigation identified an issue with Sysmon telemetry forwarding, which prevented the required Windows events from reaching the Wazuh detection engine. The configuration was investigated and corrected, after which the required Sysmon events became visible and the affected detections could be tested successfully.

The Process Discovery (Rule 7) detection also required additional troubleshooting and tuning before it generated the expected alert. The detection condition was adjusted to correctly match the available Windows command-line telemetry, after which the Atomic Red Team simulation successfully triggered the rule.

A remaining infrastructure gap is the pfSense log integration. Although pfSense traffic/log data was observed at the Wazuh manager during troubleshooting using TCPDump, the pfSense logs were not successfully onboarded and displayed as a functional log source within the Wazuh dashboard. Therefore, pfSense remains an area requiring further integration work.

These challenges demonstrate that successful SOC detection depends not only on creating detection rules but also on ensuring reliable telemetry collection, correct log forwarding, appropriate event parsing, and continuous validation of the detection pipeline.

Recommendations / Next Steps

The primary next step is to continue troubleshooting and completing the pfSense integration with Wazuh until pfSense logs are successfully onboarded and consistently displayed within the Wazuh dashboard. This will close the remaining telemetry-integration gap identified during the project.

The custom detection rules should also continue to be monitored and tested periodically to ensure that they remain effective as the laboratory environment changes. Additional controlled simulations should be performed to validate detection reliability and identify opportunities for further tuning.

The detection and triage workflow should also be practiced continuously using realistic security scenarios. This will help ensure that the detection rules, alert correlation process, severity classification, escalation decisions, and triage playbook continue to reflect real-world SOC operations.

Future improvements should therefore focus on completing the remaining log-source integration, maintaining and tuning the detection rules, expanding simulated attack scenarios, and continuously validating the complete monitoring and response workflow.

Conclusion

This project successfully extended and validated the SOC monitoring and detection capabilities developed in the previous SOC lab. The project covered four phases: validation of existing telemetry and identification of infrastructure gaps, development of custom Wazuh detection rules, validation and tuning of those detections using Atomic Red Team simulations, and practical alert triage using a structured playbook.

The testing demonstrated that the laboratory could generate and collect meaningful security telemetry from the monitored endpoints and use that telemetry to detect simulated adversary activity. Five custom detection rules were successfully validated, with one detection requiring additional troubleshooting and tuning before producing the expected alert. The investigation also demonstrated the importance of reliable telemetry collection, particularly through the identification and correction of the Sysmon forwarding issue.

The project further demonstrated the practical application of SOC alert triage. A confirmed SSH security event was investigated through correlation of multiple authentication failures with a subsequent successful authentication and was escalated based on the available evidence. A separate Windows

system alert was investigated and determined to be legitimate activity, demonstrating the ability to distinguish a false positive from a potentially malicious event.

Although the pfSense integration remains an outstanding gap, the project achieved its primary objectives of validating telemetry, developing and testing detections, tuning detection logic where required, and applying a repeatable triage workflow.

Overall, the project provided practical experience across the complete SOC detection lifecycle—from telemetry collection and detection engineering through adversary simulation, alert investigation, correlation, escalation, and false-positive handling—and established a foundation for further development of the laboratory into a more complete and realistic SOC environment.

Appendix A

Custom Wazuh Correlation Rules

The following appendix contains the final custom Wazuh correlation rules developed and validated during the project. These rules were created to detect the selected MITRE ATT&CK techniques and were subsequently tested using controlled Atomic Red Team simulations. The final configuration shown for each rule represents the version used during validation.

PowerShell Execution - MITRE T1059.001

```
<rule id="100301" level="10">
  <if_sid>61603</if_sid>
  <field name="win.system.eventID">4103</field>
  <description>Rule 3: PowerShell execution detected - MITRE T1059.001</description>
  <mitre>
    <id>T1059.001</id>
  </mitre>
</rule>
```

Scheduled Task Creation - MITRE T1053.005

```
<rule id="100302" level="10">
  <if_sid>61603</if_sid>
  <field name="win.system.eventID">4104</field>
  <description>Rule 4: Scheduled task creation detected - MITRE T1053.005</description>
  <mitre>
    <id>T1053.005</id>
```

</mitre>

</rule>

SSH Remote Service - MITRE T1021.004

<rule id="100303" level="10">

<if_group>sshd</if_group>

<description>Rule 5: SSH remote service activity detected - MITRE T1021.004</description>

<mitre>

<id>T1021.004</id>

</mitre>

</rule>

Local Account Discovery - MITRE T1087.001

<rule id="100304" level="10">

<field name="win.eventdata.commandLine"
type="pcr2">(?)\b(net\.exe\s+user|net\s+user|whoami(?:\.exe?))\b</field>

<description>Rule 6: Local account discovery command detected - MITRE T1087.001</description>

<mitre>

<id>T1087.001</id>

</mitre>

</rule>

Process Discovery - MITRE T1057

<rule id="100305" level="10">

<if_sid>61603</if_sid>

<field name="win.eventdata.commandLine">(?)tasklist(\.exe)?</field>

<description>Rule 7: Process discovery detected - MITRE T1057</description>

<mitre>

<id>T1057</id>

</mitre>

</rule>